SENA · TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE

Documento de evidencias

Frontend en React del proyecto QR Empresarial

PROYECTO	QR Empresarial — perfiles empresariales con código QR dinámico
APRENDIZ	Jean Carlos Orozco Hernández
FICHA	3336250
GUÍAS	Frontend N.º 1 y N.º 2, más los ocho módulos del proyecto
REPOSITORIO	github.com/JECAOR/qr-empresarial
VERSIÓN	v3.3.0
FECHA	31 de octubre de 2026

Todas las capturas de este documento se tomaron contra el servidor real, con los datos de `database/seed.sql`, y se regeneran con un guion versionado.

1. Conceptos fundamentales utilizados

El frontend es una aplicación de una sola página construida con **React 19** y **Vite 8**, que consume la API REST del propio proyecto. No accede nunca a la base de datos: pide y envía datos por HTTP, y es el backend quien decide qué se puede hacer.

Componentes, props y composición

Cada pieza de interfaz es una función que devuelve JSX. Lo que cambia entre usos llega por **props**: `TarjetaContenido` recibe el título, el detalle y el tipo de bloque, y decide con una condición si se dibuja como enlace o como texto. Así la misma pieza sirve para los seis bloques del perfil sin repetir código.

Estado y efectos

`useState` guarda lo que cambia mientras la pantalla está abierta y avisa a React de que debe volver a dibujar. `useEffect` sirve para lo que ocurre fuera del dibujado —pedirle los datos al servidor— y su lista de dependencias decide cuándo se repite: con `[dias]`, solo al cambiar el periodo de las estadísticas.

Cada efecto que pide datos devuelve una función de limpieza. Sin ella, si se consulta un negocio y antes de que llegue la respuesta se consulta otro, la primera podría pisar a la segunda.

> **Dos peticiones en desarrollo no son un error.** `StrictMode` ejecuta cada efecto dos veces a propósito, para que salte a la vista un efecto que no sabe limpiarse. En producción se ejecuta una sola vez.

Contexto

La sesión —quién eres, tu rol, entrar y salir— vive en un **contexto** (`AuthContext`) en lugar de pasarse por props de pantalla en pantalla. Al abrir la aplicación no se confía en lo que hay guardado en el navegador: se le pregunta al servidor con `GET /usuarios/me`, de modo que un token caducado o revocado se detecta enseguida.

Enrutado y rutas protegidas

React Router decide qué pantalla se dibuja según la dirección. `RutaProtegida` exige sesión y, cuando hace falta, un rol.

> **El guardia no es seguridad.** Quien escriba la dirección a mano o llame a la API con `curl` se lo salta entero. La seguridad la aplica Express en cada endpoint: una cuenta de empresa recibe 403 aunque consiga pintar la pantalla. Lo que hace el guardia es no ofrecer puertas que van a dar error.

Separación en capas

Ninguna pantalla llama a `fetch`. Todas pasan por un servicio, y todos los servicios por un único `apiClient` que pone la dirección base, el tipo de contenido y el token, y convierte un error del servidor en una excepción con su mensaje. Si mañana cambia la forma de autenticarse, se toca un archivo.

2. Componentes implementados y su justificación

2.1 Estructura del proyecto

```
src/
  context/AuthContext.jsx        La sesión: quién eres, entrar, salir y refrescar
  components/
    Layout/                      Barra de navegación común
    RutaProtegida/               Exige sesión y, si hace falta, un rol
    Formulario/CampoTexto        Etiqueta, campo y ayuda, en una sola pieza
    FichaNegocio/                Datos del negocio; omite los campos vacíos
    TarjetaContenido/            Un bloque: enlace o texto, según su tipo
    contenido/ListaBloques       Los bloques con su orden y sus acciones
    contenido/FormularioBloque   Alta y edición de un bloque
    usuarios/TablaUsuarios       El listado; en móvil se vuelve fichas
    usuarios/FormularioUsuario   Alta y edición de una cuenta
    perfil/SelectorTema          El color de marca
    estadisticas/GraficaActividad La gráfica diaria, en SVG
    admin/DetalleCuenta          La actividad reciente de una cuenta
  pages/                         Trece pantallas
  services/                      Once servicios, uno por módulo de la API
  pruebas/                       Lo que montan las 417 pruebas

Cada archivo .test.jsx vive al lado de lo que prueba, no en una
carpeta aparte: asi se ve de un vistazo que tiene pruebas y que no.
```

2.2 Por qué cada decisión

COMPONENTE	JUSTIFICACIÓN
apiClient	Única puerta de salida hacia la API. El ejemplo de la guía comprueba body.success; esta API no tiene ese campo, y fiarse de él haría que todo pareciera correcto. Manda el código HTTP.
AuthContext	La sesión la necesitan la barra, el guardia y cuatro pantallas. Pasarla por props obligaría a atravesar componentes que no la usan.
RutaProtegida	Un solo sitio donde se decide quién pasa. Recuerda a dónde iba el usuario para devolverlo ahí después de entrar.
CampoTexto	Las cuatro pantallas de cuenta repetían el mismo bloque de etiqueta, campo y texto de ayuda.
GraficaActividad	SVG dibujado a mano, sin librería: son dos series de entre siete y noventa puntos, y traer chart.js con react-chartjs-2 costaría 70 KB comprimidos —la aplicación pasaría de 98 a 169, un 72 % más—, medido instalándolos y compilando. El <title> de cada punto da la información al pasar el cursor, que es cosa del navegador.
DetalleCuenta	Va en un <dialog> para que el navegador se encargue del foco, de Escape y de bloquear lo de detrás.
TablaUsuarios	Bajo 780 px cada fila se convierte en una ficha con el nombre de la columna delante de su valor. Una tabla de siete columnas en un móvil obliga a desplazar la pantalla a lo ancho para alcanzar los botones.

2.3 Dependencias

Tres en producción y once de desarrollo, seis de ellas solo para las pruebas. **Ninguna librería de interfaz**: los estilos, la gráfica y los diálogos son propios.

PAQUETE	VERSIÓN	PARA QUÉ
react / react-dom	19.3	La biblioteca y su puente con el navegador
react-router-dom	7.18	Enrutado y rutas protegidas
vite	8.3	Servidor de desarrollo y empaquetado
@vitejs/plugin-react	6.1	Compilación de JSX y recarga en caliente
oxlint	1.88	Análisis estático
vitest	3.2	El motor de las pruebas; usa la misma configuración que Vite
@testing-library/react	16.3	Monta un componente y lo consulta como lo haría una persona
jsdom	30.1	El navegador de mentira donde se dibujan

Las de desarrollo no entran en lo que se descarga el visitante. Resultado: **103 KB comprimidos** para trece pantallas —98 de JavaScript y 5 de estilos—, medidos con `npm run build`.

3. Evidencia de integración con la API

El frontend consume **39 de los 40 endpoints** de la API REST del proyecto. El que falta es `GET /api/health`, que solo comprueba que el servidor responde.

`POST /api/publico/:slug/eventos` —el que cuenta escaneos, visitas y clics— lo llama **solo** la página del código QR, `/p/:slug`. El panel del dueño no, o inflaría sus propias estadísticas cada vez que abriera su perfil. La diferencia no se ve en pantalla: equivocarse ahí no rompe nada, solo deja las cifras mintiendo, y por eso tiene pruebas propias.

3.1 Endpoints por módulo

MÓDULO	ENDPOINTS	PANTALLA
Autenticación	/auth/login, /logout, /registro, /recuperar-contrasena, /restablecer/:token, /restablecer-contrasena	Entrar, Registro, Recuperar, Restablecer
Cuenta propia	/usuarios/me (GET, PUT, DELETE), /me/contrasena, /me/preferencias (GET, PUT)	Configuración
Usuarios	/usuarios (GET, POST), /:id (GET, PUT, DELETE), /:id/estado	Gestión de usuarios
Perfil	/perfiles/me (GET, PUT), /me/tema	Mi negocio
Contenido	/contenido (GET, POST), /categorias, /orden, /:id (PUT, DELETE), /:id/estado	Contenido
Código QR	/qr (GET, POST), /estado, /imagen	Código QR
Vista pública	/publico/:slug, /:slug/eventos	Perfil público, página del QR
Estadísticas	/estadisticas	Actividad
Administración	/admin/resumen, /admin/usuarios, /:id, /reportes/usuarios	Administración

3.2 Lo que hubo que adaptar del ejemplo de las guías

Las guías usan SICE como proyecto orientador y advierten que cada equipo debe comprobar el contrato real de su API. Al hacerlo aparecieron ocho diferencias, todas comprobadas contra el servidor con `curl` antes de escribir una sola pantalla:

EL EJEMPLO ASUME	ESTA API RESPONDE
Campo success en el cuerpo	No existe: manda el código HTTP
data.user y expiresIn	data.usuario, sin expiresIn
role como objeto {id, name}	rol como cadena
{email, password} al entrar	{correo, contrasena}
Cuatro roles	Dos: empresa y administrador
/users/:id/status	/usuarios/:id/estado
Estado como 1 o 0	Como palabra: activo o restringido
El CRUD abierto a varios roles	Solo administrador

> Copiar el ejemplo tal cual habría dado `contenido.map is not a function` y una pantalla sin rol con la que no se puede proteger nada. La guía lista ese error entre los frecuentes.

3.3 Comprobación de las peticiones

La colección de Postman del proyecto trae dos carpetas dedicadas a este frontend, con **46 peticiones y 44 pruebas automáticas** que recorren los mismos endpoints en el mismo orden que las pantallas, incluidos los casos negativos —400, 401, 403, 404 y 409— y el contraste de permisos entre un administrador y una empresa.

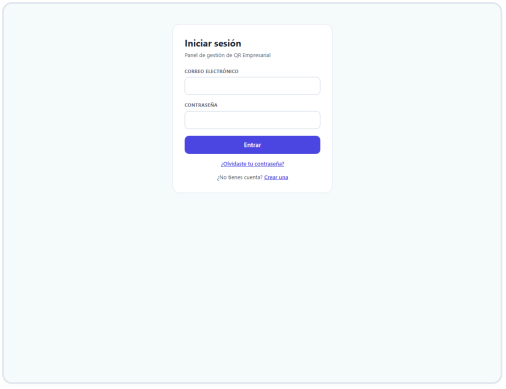
```
Guia React — Frontend N.1    3 peticiones   13 pruebas
Guia React — Frontend N.2   15 peticiones   31 pruebas
```

Crean su propia cuenta de prueba y la eliminan al terminar, de modo que dejan la base como la encontraron.

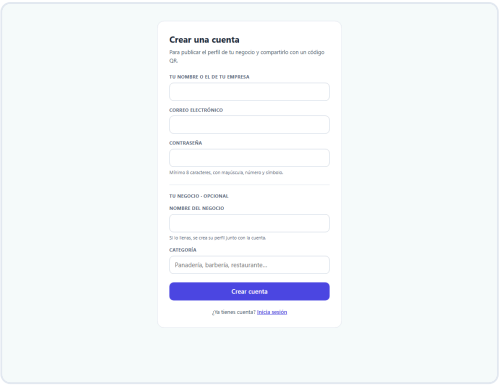
4. Capturas de la interfaz

Las dieciséis capturas siguientes las genera `frontend-react/evidencias/generar-capturas.js`, que abre un navegador, entra con las cuentas de demostración y recorre la aplicación **contra el servidor real**. Lo que se ve sale de MySQL: una pantalla dibujada con datos inventados no demuestra nada.

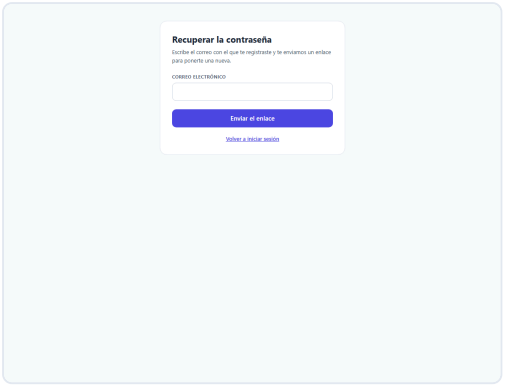
4.1 Acceso y cuenta



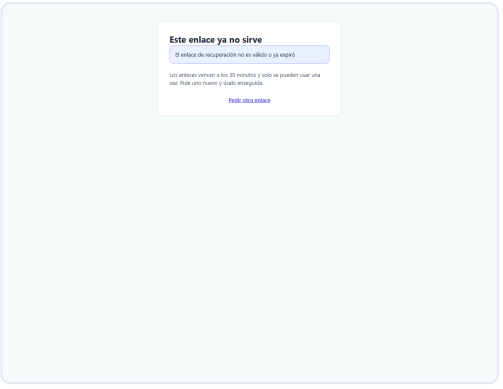
01 · Iniciar sesión. Guarda el token en `sessionStorage`: la sesión muere al cerrar la pestaña.



02 · Registro. El negocio es opcional; si se llena, la cuenta y su perfil se crean en una misma transacción.

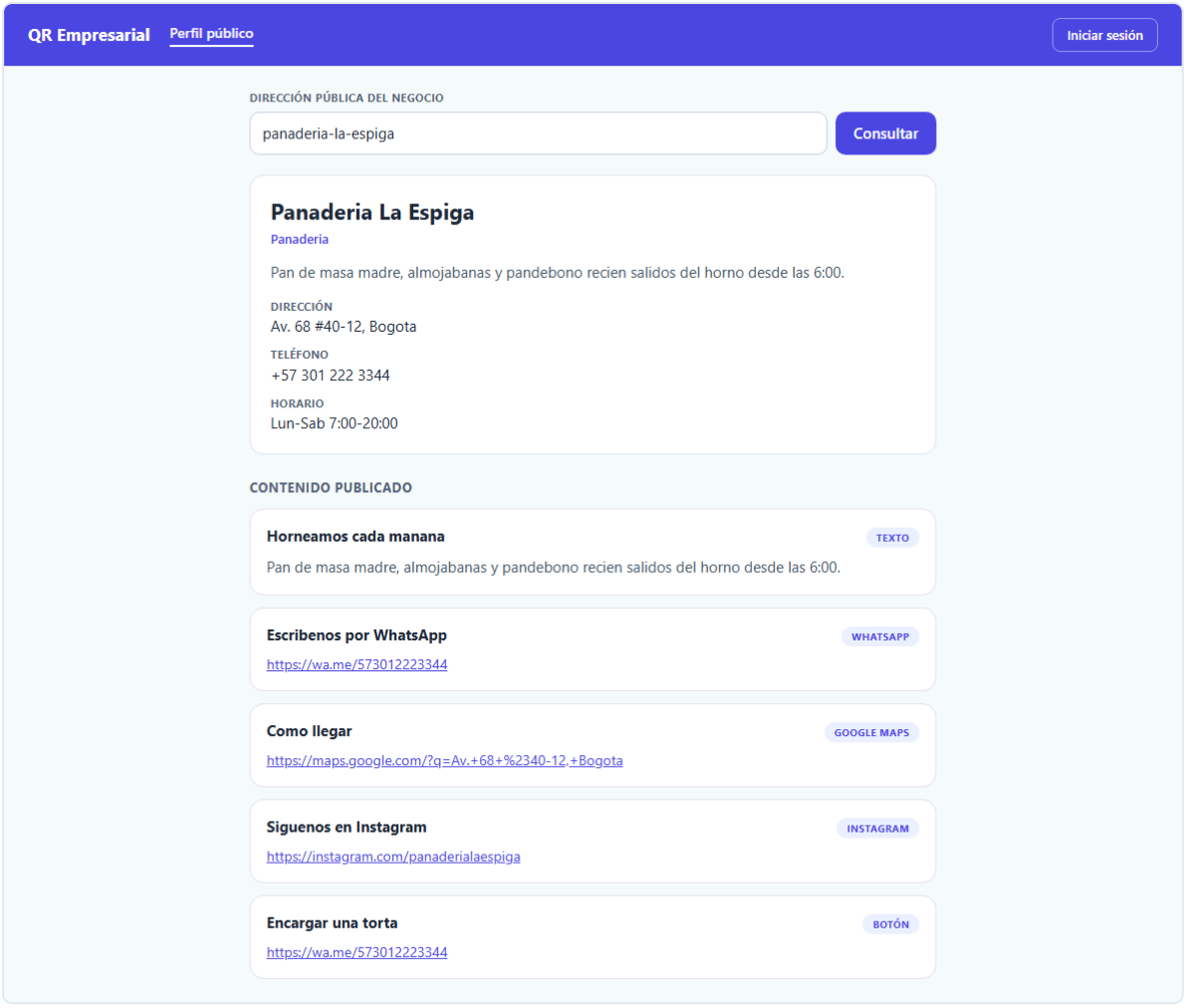


03 · Recuperar la contraseña. Responde lo mismo exista o no el correo, para no revelar qué direcciones están registradas.

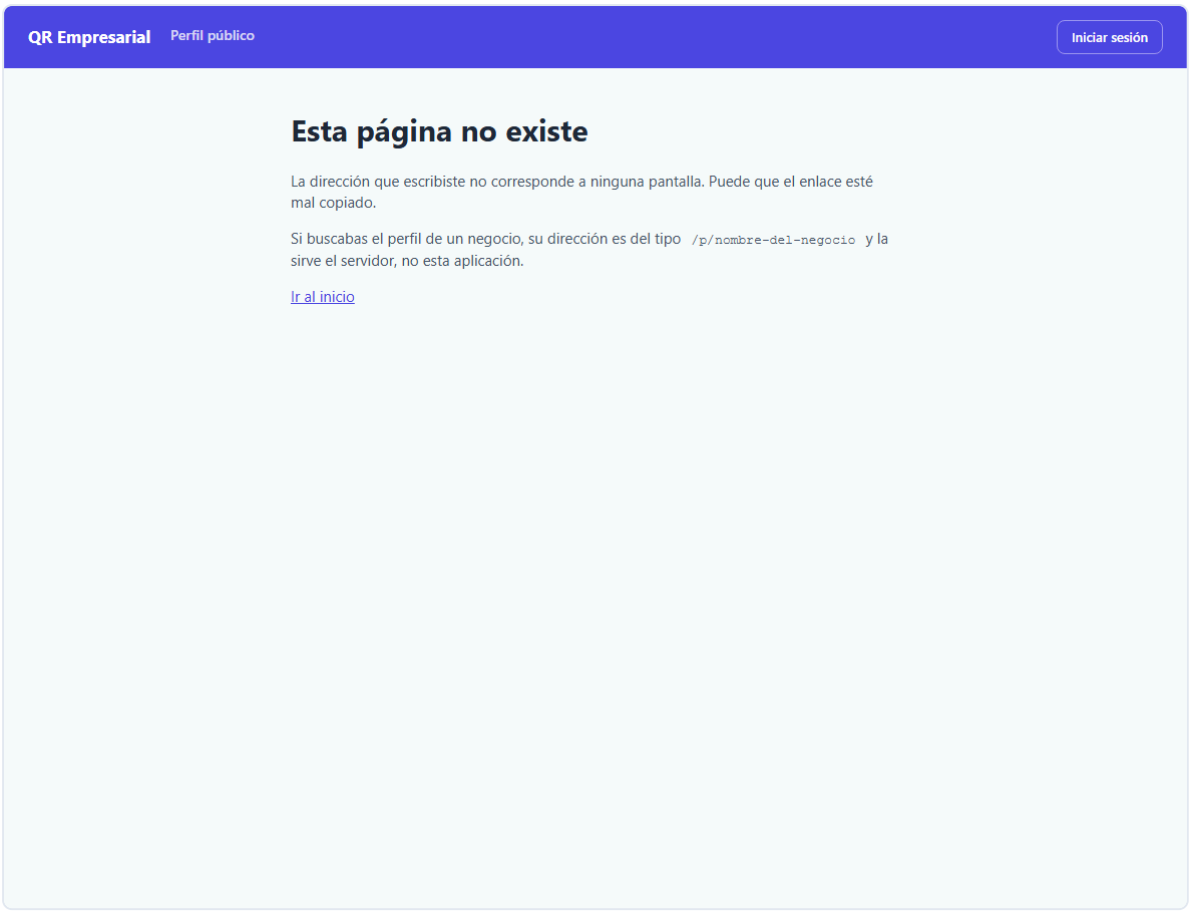


04 · Enlace caducado. Se comprueba antes de enseñar el formulario: pedir una contraseña nueva para luego decir que el enlace no sirve es la peor manera de contarlo.

4.2 Vista pública y errores

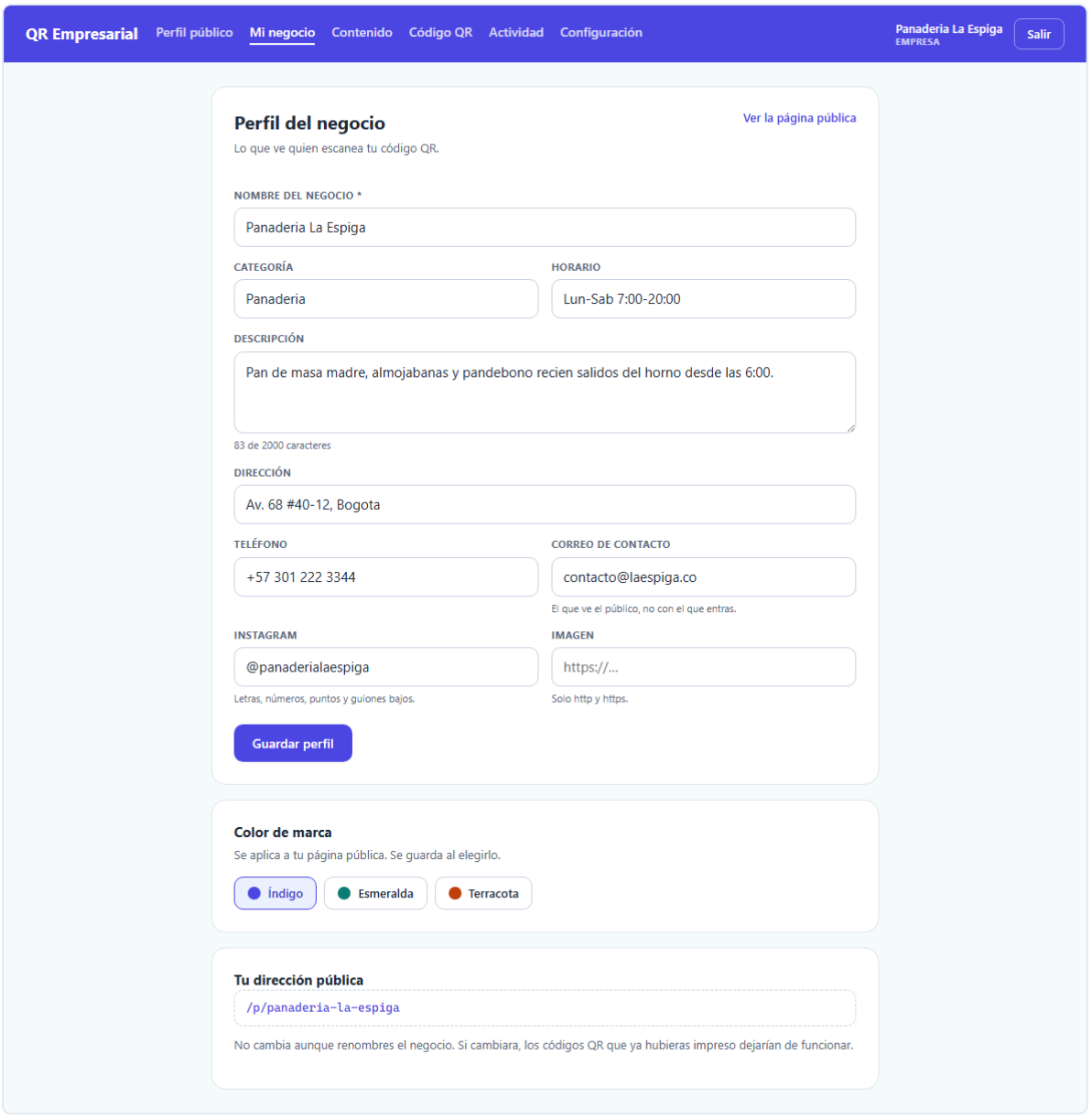


05 · Perfil público. Lo que ve quien escanea el código. No pide sesión. Solo aparecen los bloques activos: el dueño ve seis, el público cinco.

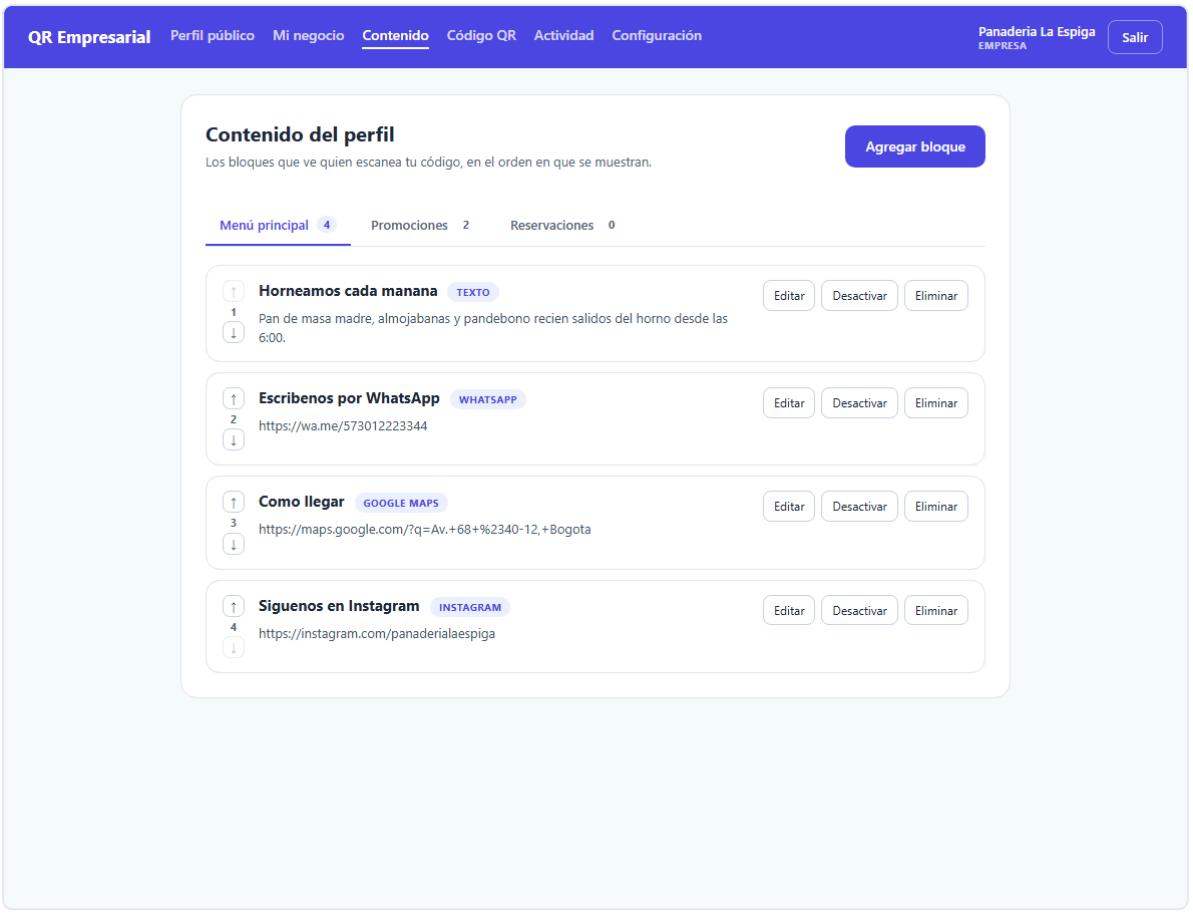


06 · Dirección inexistente. Se dice en lugar de redirigir a la portada en silencio, que haría creer al usuario que llegó donde quería.

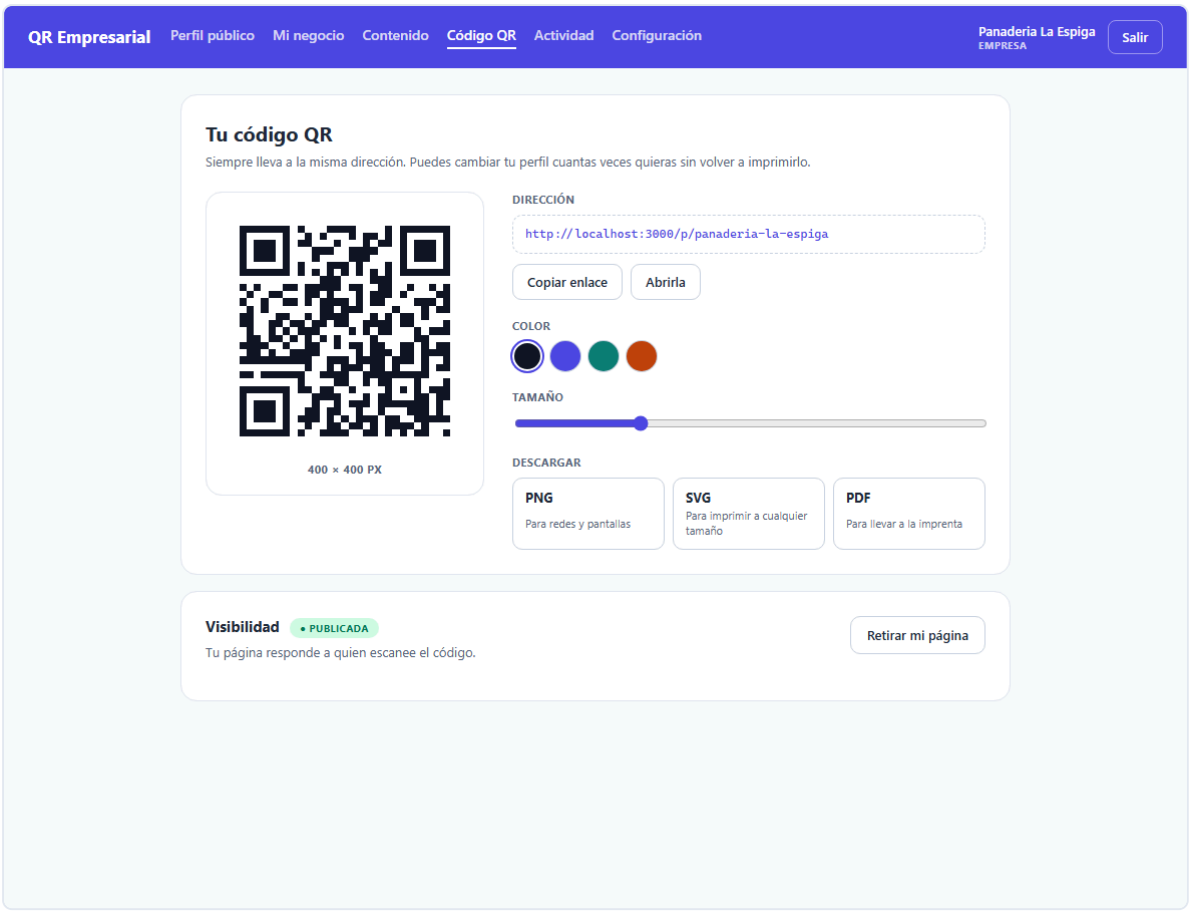
4.3 Panel del negocio



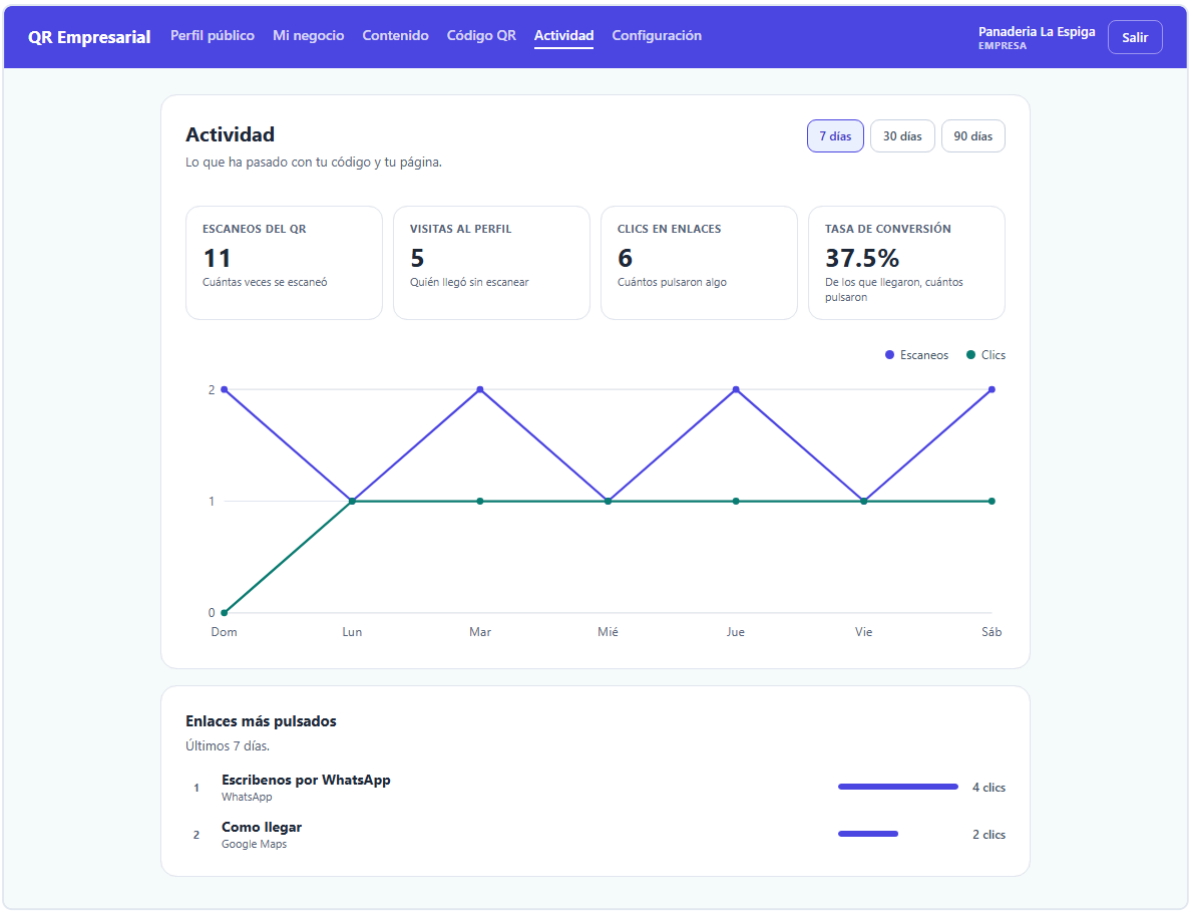
07 · Mi negocio. Los nueve campos del perfil, el color de marca y la dirección pública, que se muestra pero no se edita: si el slug cambiara, los códigos ya impresos dejarían de funcionar.



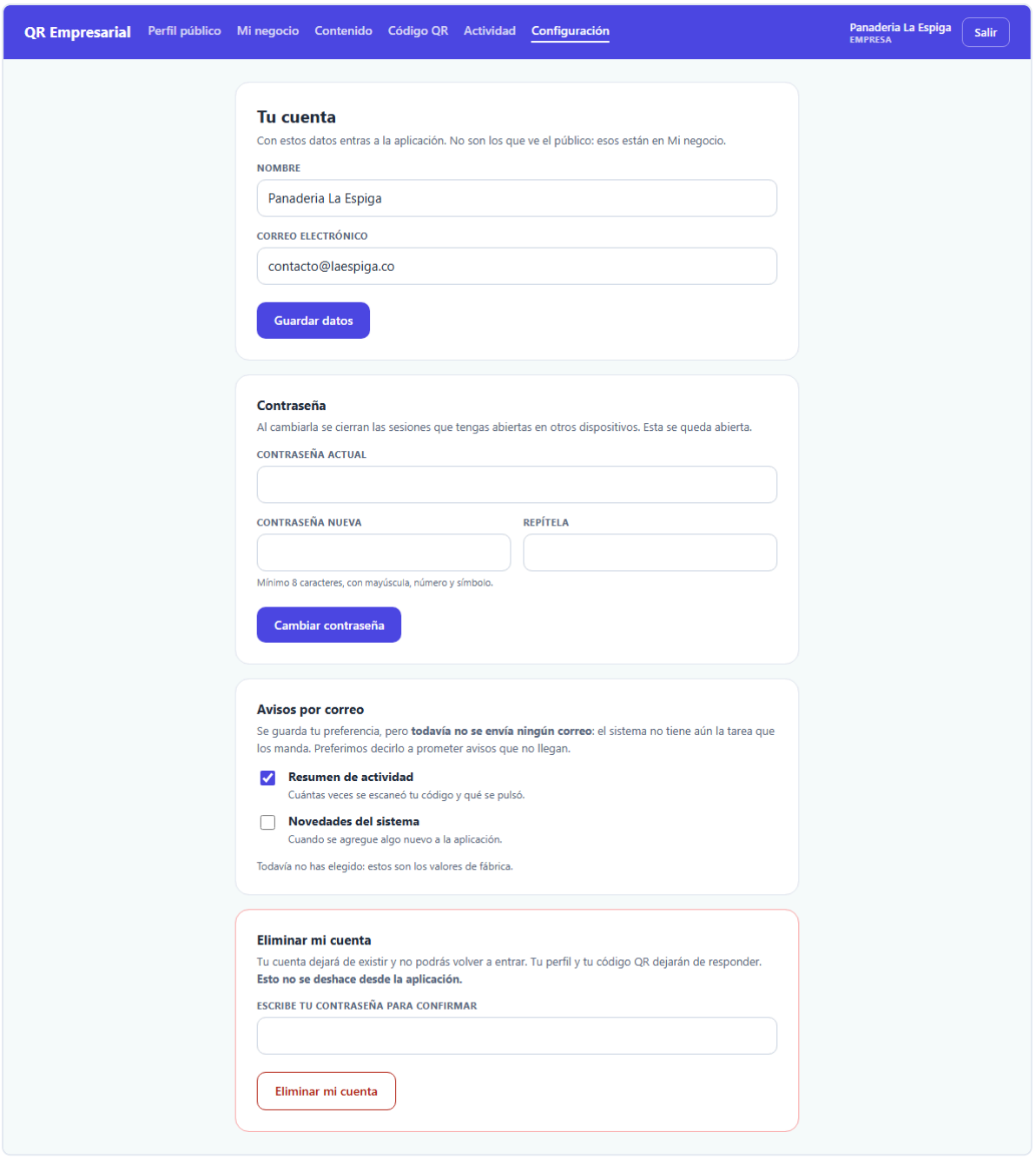
08 · Contenido. Tres secciones con su cuenta de bloques. Se reordena arrastrando y con las flechas: el arrastre de HTML no funciona con el dedo, y sin las flechas un móvil se quedaría sin forma de reordenar.



09 · Código QR. La vista previa la genera el servidor, el mismo endpoint que la descarga: lo que se ve es exactamente el archivo que se va a bajar.

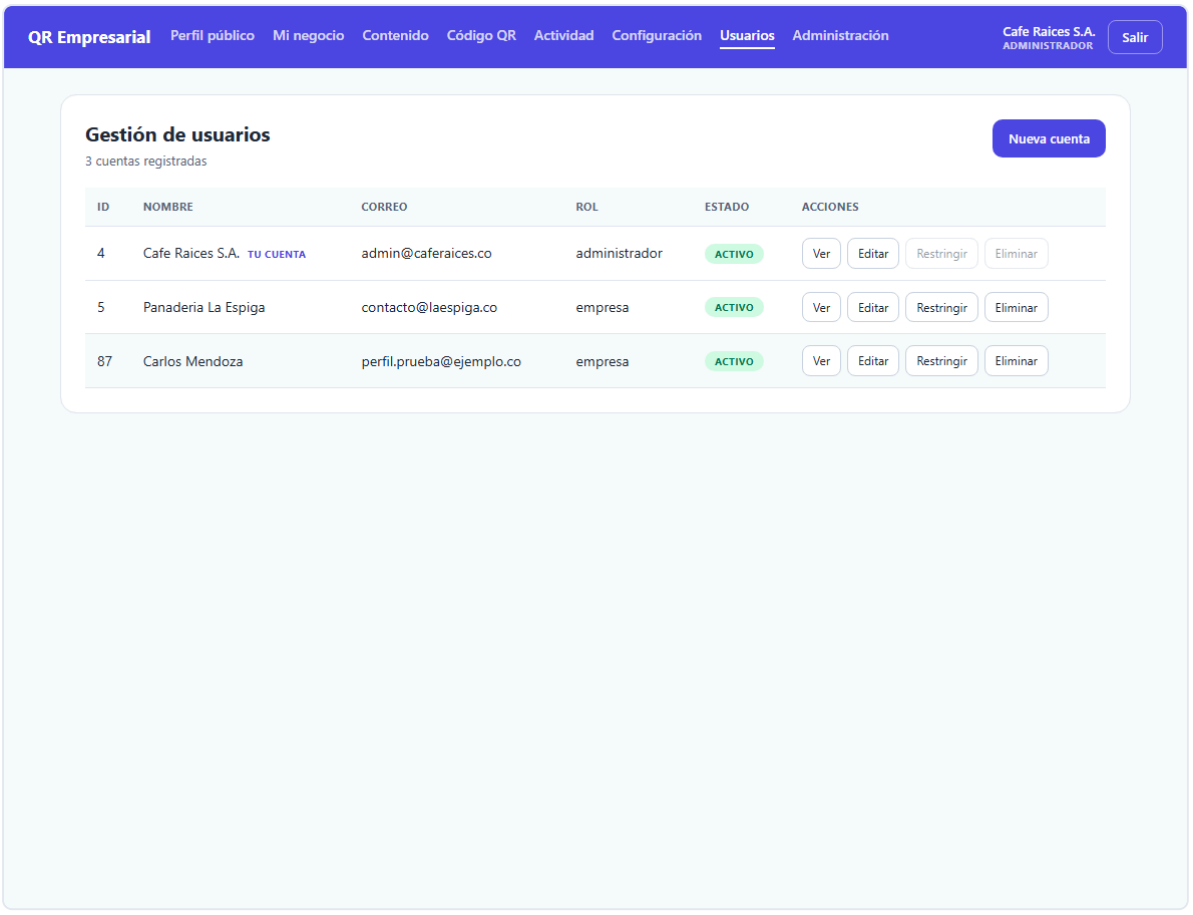


10 · Actividad. Cuatro medidas, la gráfica diaria en SVG y los enlaces más pulsados, en 7, 30 o 90 días —los tres rangos que acepta el servidor—.

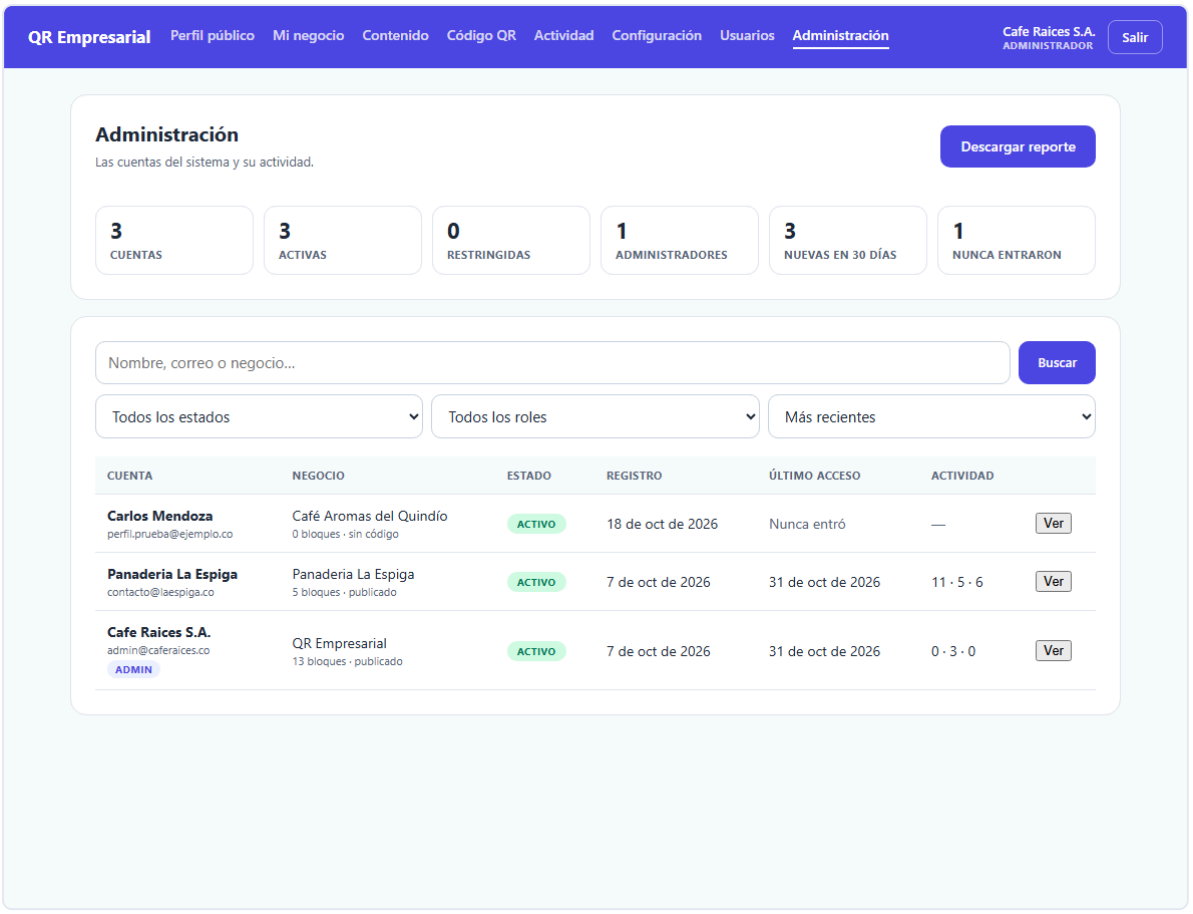


11 · Configuración. Datos, contraseña, avisos y baja. La contraseña solo se pide cuando cambia el correo. Los avisos se guardan pero no se envían, y la pantalla lo dice.

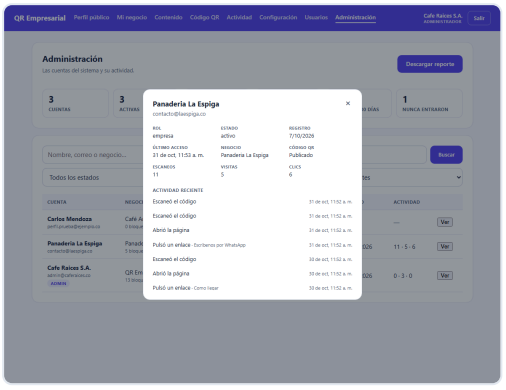
4.4 Administración



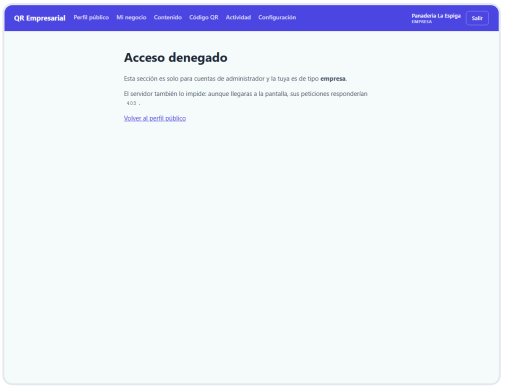
12 · Gestión de usuarios. Los siete endpoints del módulo. Restringir y eliminar están deshabilitados en la propia fila: el servidor lo impide y ofrecer un botón que solo puede fallar es una mala pantalla.



13 · Administración. Seis cifras, filtros con búsqueda, paginación y el reporte en CSV, que respeta los filtros y empieza por BOM para que Excel no rompa las tildes.

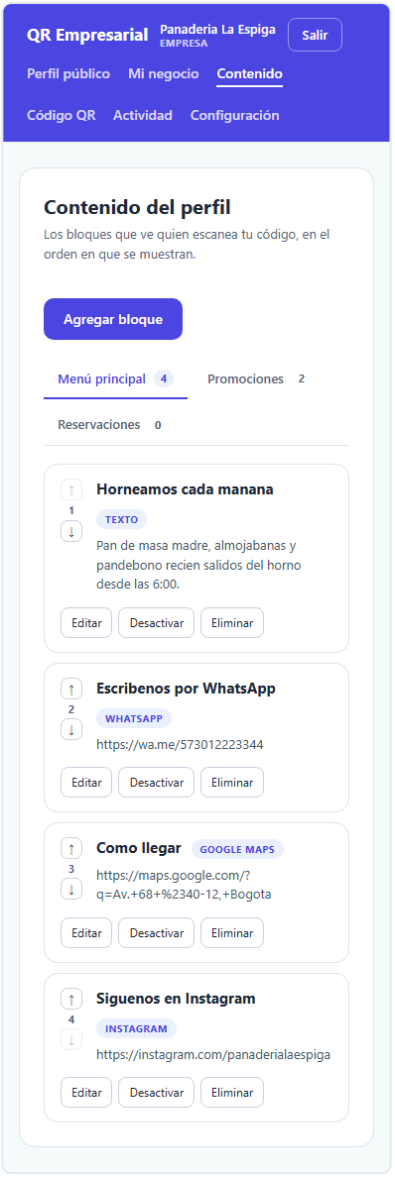


14 · Detalle de una cuenta. La actividad reciente, que la tabla no cabe.



15 · Acceso denegado. Una cuenta de empresa en una ruta de administrador.

4.5 Diseño adaptable



16 · Contenido a 390 px. Sin desplazamiento horizontal: la navegación baja a su propia línea y las acciones de cada bloque se reparten.

5. Repositorio Git

URL	https://github.com/JECAOR/qr-empresarial
Código del frontend	frontend-react/
README	frontend-react/README.md y el de la raíz
Evidencias	frontend-react/evidencias/

5.1 Historial de commits del frontend

Veinte commits sobre `frontend-react/`, del 21 al 31 de octubre de 2026, dentro de un repositorio con **126 commits y 50 etiquetas**. Cada módulo se desarrolló en su propia rama y se integró con `merge --no-ff`, de modo que la rama queda en el historial en lugar de aplanarse.

```
2026-10-31  test(frontend-react): 417 pruebas propias con Vitest         v3.2.0
2026-10-31  docs(evidencias): el documento tambien en Word               v3.1.0
2026-10-31  docs(evidencias): documento de evidencias del frontend       v3.0.1
2026-10-23  feat(frontend-react): el modulo 8, la administracion         v3.0.0
2026-10-23  feat(frontend-react): el modulo 7, la cuenta                 v2.12.0
2026-10-23  feat(frontend-react): el modulo 6, la actividad del perfil   v2.11.0
2026-10-23  feat(frontend-react): el modulo 4, el codigo QR              v2.10.0
2026-10-23  feat(frontend-react): el modulo 3, el contenido del perfil   v2.9.0
2026-10-22  feat(frontend-react): el modulo 2, el perfil del negocio     v2.8.0
2026-10-22  feat(frontend-react): el modulo 1 completo                   v2.7.0
2026-10-22  docs(evidencias): las tres capturas que cierran la Guia 02   v2.6.0
2026-10-22  test(postman): carpeta de la Guia Frontend N.2               v2.5.1
2026-10-22  docs(evidencias): matriz de pruebas y capturas de la Guia 02
2026-10-22  feat(frontend-react): sesion JWT, rutas por rol y usuarios   v2.5.0
2026-10-21  docs(evidencias): la captura de Vite, Guia 01 completa       v2.4.0
2026-10-21  docs(evidencias): las cinco primeras capturas de la Guia 01
2026-10-21  docs(evidencias): Postman es una sola captura, no dos
2026-10-21  docs(frontend-react): carpeta de evidencias de la Guia 01
2026-10-21  docs(frontend-react): cierra los tres huecos de la Guia 01   v2.3.1
2026-10-21  feat(frontend-react): primera pantalla en React, Guia N.1    v2.3.0
```

Los mensajes siguen **Conventional Commits** y explican el porqué, no solo el qué. El historial completo está en GitHub.

5.2 Cómo ejecutarlo

```
mysql -u root -p --default-character-set=utf8mb4 qr_empresarial < database/seed.sql

cd backend        && npm install && npm run dev    # http://localhost:3000
cd frontend-react && npm install && npm run dev    # http://localhost:5173
```

Las pruebas del frontend no necesitan ninguna de las dos cosas: ni base de datos ni servidor.

```
cd frontend-react && npm test          # las 417
cd frontend-react && npm run coverage  # con el informe de cobertura
```

Cuentas de demostración, las dos con la contraseña `Clave123*`: `admin@caferaices.co` (administrador) y `contacto@laespiga.co` (empresa).

5.3 Defectos encontrados y corregidos

Construir estas pantallas destapó cinco defectos, cuatro en el propio frontend y uno en el backend. Se documentan porque forman parte del proceso:

DEFECTO	CAUSA Y CORRECCIÓN
Un botón se publicaba como texto (backend)	El tipo se decidía comparando el nombre de la categoría con el literal 'Botón', y el cliente mysql de la consola de Windows reescribía los acentos al cargar las migraciones. Se corrigió con SET NAMES utf8mb4 en los dieciséis archivos SQL y comparando por el icono, que es ASCII.
Nombres de columnas en la API (backend)	GET /usuarios devolvía las filas tal como salen de MySQL mientras el resto respondía en camelCase. Nadie lo veía porque el frontend original no llama a esa ruta.
El diálogo se cerraba solo	close() dispara el evento close, que borraba el estado; y StrictMode ejecuta el efecto dos veces. La limpieza sobraba.
El arrastre fallaba al ser rápido	El índice de origen iba en estado de React, que no se ha aplicado cuando llega el drop si los tres eventos caen en el mismo tic. Se pasó a useRef.
La barra se quedaba con el nombre viejo	Al cambiar el nombre, la barra seguía mostrando el anterior hasta recargar. AuthContext gana un refrescar().

5.4 Las pruebas automáticas

439 pruebas en 26 archivos, que tardan medio minuto. Se ejecutan con `npm test` dentro de `frontend-react/`. El motor es **Vitest**, que usa la misma configuración que Vite, con **Testing Library** sobre **jsdom**: cada componente se monta de verdad y se consulta como lo haría una persona —por su etiqueta, por su texto, por su papel—, no por el nombre de sus clases CSS.

QUÉ SE PRUEBA	PRUEBAS	LO QUE SE COMPRUEBA
Las quince pantallas	194	El recorrido entero: cargar, escribir, guardar, fallar y reintentar
Los doce componentes	108	Lo que dibujan con cada dato y con qué avisan al pulsarlos
Los once servicios	90	La ruta, el método y el cuerpo de cada llamada, y qué pasa con cada error
El mapa de rutas	28	Que cada dirección lleve a su pantalla y que las protegidas lo estén
El contexto de sesión	11	Que no se fíe del navegador y que tire la sesión si el servidor la rechaza
El guardia de rutas	8	Quién pasa, a dónde se le manda y que no se enseñe nada mientras comprueba

Cobertura: 99,4 % de las líneas y 96,4 % de las ramas (`npm run coverage`). Ninguna prueba llega a la red: `.env.test` apunta a un servidor que no existe, de modo que una que se escapara fallaría en lugar de tocar la base de datos de desarrollo.

5.5 Cada prueba se falsificó

Una prueba que pasa no demuestra nada si también pasaría con el código roto. Los cinco defectos de la sección anterior se volvieron a meter, uno a uno, para comprobar que su prueba fallaba:

DEFECTO REINTRODUCIDO	QUÉ PASÓ
La fecha de la gráfica leída con new Date()	La etiqueta pasó a decir «30 sep» en lugar de «1 oct»: el mismo día de menos que se vio en pantalla
La limpieza que cerraba el diálogo	Dos pruebas en rojo, incluida la que lo monta en StrictMode
El login que mandaba a todos a /usuarios	Dos pruebas en rojo
Las preferencias que ignoraban al servidor	La nota de «valores de fábrica» no desaparecía: una prueba en rojo
Quitar el refrescar() de la cuenta	Una prueba en rojo
Fiarse del campo success del ejemplo	Diez de las dieciocho de apiClient en rojo

5.6 Dos cosas que estas pruebas no demuestran

Se dicen porque callarlas sería justamente el tipo de error que estas pruebas existen para evitar.

El arrastre rápido no se puede reproducir aquí. El defecto era una carrera: los tres eventos del arrastre caen en el mismo tic y el estado de React todavía no se ha aplicado. Pero `fireEvent` envuelve cada evento en `act()`, que vacía la cola de React, así que la carrera no llega a darse: se comprobó volviendo a poner el

`useState`, y las dieciséis pruebas seguían en verde. En su lugar se mide con un `Profiler` que empezar un arrastre no provoque un redibujado, que es lo que distingue una `ref` del estado, y esa sí falla con el código viejo.

El destino recordado al entrar tampoco. Por el mismo `act()`, el `<Navigate>` de «ya tienes sesión» le gana al `navegar(desde)` y la pantalla acaba en la portada. Se comprobó en un Chromium de verdad, contra los dos servidores: entrar en `/contenido` sin sesión e iniciarla aterriza en `/contenido`. La prueba afirma por eso a dónde manda la pantalla, no dónde acaba.

5.7 El frontend publicado

Desde la versión **v3.3.0** es este el frontend que sirve el servidor: abrir la dirección del proyecto entrega esta aplicación, y los códigos QR llevan a su página `/p/:slug`. El original sigue en el repositorio y se publica cambiando una variable, `FRONTEND=clasico`; sus 396 pruebas de Playwright pasan igual, porque levantan su propio servidor.

Publicarlo obligó a tres cosas que no hacían falta con el original, y que son las que de verdad distinguen servir una aplicación de una sola página:

QUÉ HUBO QUE HACER	POR QUÉ
Reenviar las rutas al index.html	/perfil o /contenido no son archivos: los resuelve el enrutador en el navegador. Sin esto funcionan al navegar y dan 404 al recargar, que es el fallo más fácil de dejar pasar. Solo se reenvían los GET sin extensión: a un POST a una ruta que no existe hay que decirle que no existe, y a un .js que falta, también.
Inyectar las etiquetas de vista previa	WhatsApp, Telegram y Google no ejecutan JavaScript, y esta pantalla se dibuja entera en el navegador: sin inyectarlas en el servidor no habría nada que leer al pegar el enlace. El index.html lleva las mismas marcas que usaba la pantalla del frontend original.
Mover el enlace del correo	React no sirve archivos .html: la recuperación de contraseña apunta ahora a /restablecer. La dirección sale de la configuración y sigue a la variable, porque escribirla en el servicio la habría dejado desfasada al cambiarla.

Y una pantalla nueva, `/p/:slug`, que es la que ve quien escanea: sin buscador —ya llegó a su negocio— y con el registro de escaneos, visitas y clics que hasta ahora hacía el frontend original.

> **Un defecto encontrado al probarlo.** La primera versión guardaba en una `ref` que ya había contado la visita, para que `StrictMode` no la contara dos veces. Sobraba —la bandera de vigencia del efecto ya lo impedía— y además dejaba sin contar la visita al ir de un negocio a otro dentro de la aplicación, porque React Router reutiliza el componente cuando solo cambia el slug. La prueba que lo destapó falla con la `ref` y pasa sin ella.

5.8 Lo que este frontend todavía no tiene

No hay pruebas de extremo a extremo propias, en un navegador de verdad. El frontend original del proyecto tiene 396 de Playwright y este se verificó pantalla por pantalla a mano contra el servidor real, pero las dos comprobaciones de la sección anterior enseñan dónde está el límite de jsdom: lo que depende del momento exacto en que React aplica un cambio pide un navegador.